Tel Aviv University
The Iby and Aladar Fleischman Faculty of Engineering

05124710 – Laboratory in FPGA

Experiment 7 – Final Report

Snake Game

Authors

Name	**ID**
Saleh Khalil	213310485
Mahmood Isteteh	214032682

July 9, 2026

Contents

1 Introduction

In this experiment we designed and implemented a complete *Snake* game on the Digilent Basys3 board. The game is displayed on a computer monitor through the VGA interface, the current score is shown on the on-board 4-digit 7-segment display, and the player steers the snake with a PS/2 keyboard.

1.1 Game rules

The playing field is a 100×75 grid of 8×8-pixel cells on an 800×600 VGA screen. The snake advances one cell per game tick in the current direction; eating the red food cell makes it grow by one cell and increments the score, and every food item also makes the game tick slightly faster. The game ends when the snake hits a wall or its own body, after which a game-over screen is shown and the game can be restarted by pressing any key on the keyboard.

2 Design Overview

2.1 High-level block diagram

Figure 1 shows the complete desig, dataflow between the blocks and the three testbenches.

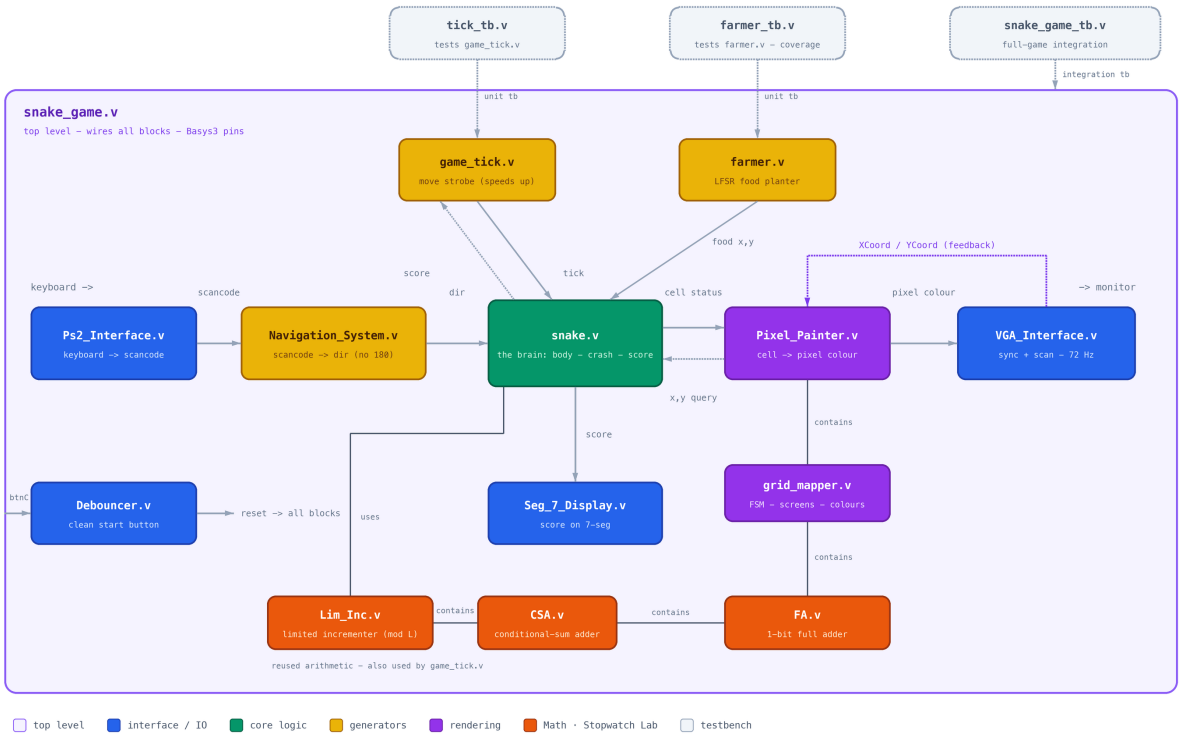


Figure 1: Full design map

2.2 File inventory

Table 1: All files and their modules

File	Module	Role	Origin
FA.v	FA	1-bit full adder	reuse (Lab 1)
CSA.v	CSA	conditional sum adder	reuse (Lab 1)
Debouncer.v	Debouncer	signal debounce	reuse (Lab 1)
Seg_7_Display.v	Seg_7_Display	score display	reuse (Lab 1)
Lim_Inc.v	Lim_Inc	limited incrementer	reuse (Lab 2)
Ps2_Interface.v	Ps2_Interface	keyboard interface	reuse (Lab 5)
VGA_Interface.v	VGA_Interface	VGA interface	reuse (Lab 6)
game_tick.v	Game_Tick	movement speed	new
snake.v	Snake	game core	new
farmer.v	farmer	food generator	new
Pixel_Painter.v	Pixel_Painter	rendering	new
grid_mapper.v	GridMapper	grid rendering	new
snake_game_tb.v	snake_game_tb	integration testbench	new
farmer_tb.v	farmer_tb	farmer testbench	new
tick_tb.v	tick_tb	dynamic tick testbench	new
Navigation_System.v	Navigation_System	direction control	new
snake_game.v	snake_game	top level	new
task3.xdc	-	Constraints	new

3 Top-Level Integration

3.1 snake_game.v – top level

Purpose. Top module file: it holds no logic of its own, only the module instantiations and the wires all modules together.

Table 2: snake_game ports

Port	Dir	Width	Description
clk	in	1	100 MHz system clock (pin W5).
rst	in	1	Centre button, active high (U18).
PS2Clk	in	1	Keyboard clock (C17).
PS2Data	in	1	Keyboard data (B17).
vgaRed/Green/Blue	out	3×4	VGA color channels.
Hsync, Vsync	out	1	VGA synchronisation.
a_to_g, an, dp	out	7/4/1	7-segment display.

3.2 IO modules - under `snake_game.v`

Of the eight instantiated modules, four are dedicated to moving data on and off the board: the keyboard, the VGA monitor, the 7-segment display and the reset button. The remaining four hold the actual game logic and are covered in the next subsection.

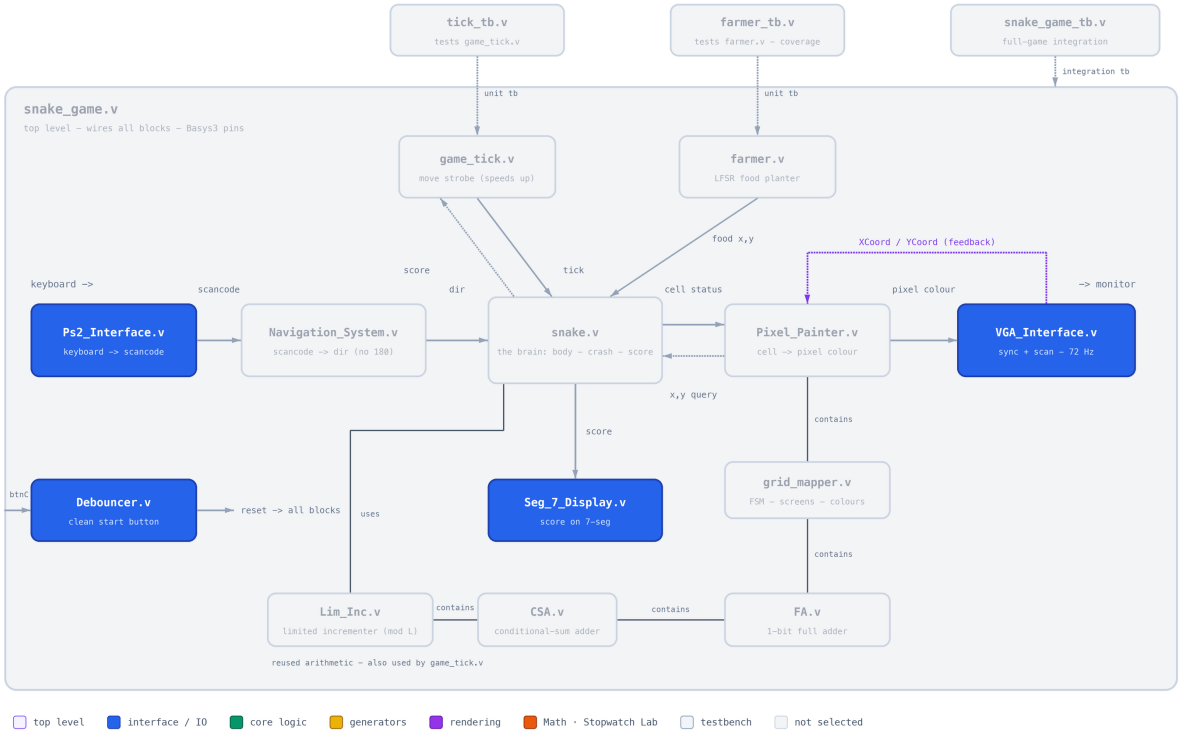


Figure 2: IO modules highlighted.

- `Ps2_Interface.v` – Receives PS/2 keyboard frames and outputs the make scancode with a single-clock `keyPressed`.
- `Debouncer.v` – Converts the bouncy centre push-button into a single one-clock reset pulse.
- `VGA_Interface.v` – Generates VGA signals for 800×600 @ 72 Hz display, drives the RGB pins with the color supplied by the `Pixel_Painter.v` module, and exports the scan coordinates.
- `Seg_7_Display.v` – Shows the 2-digit score supplied by the `Snake.v` module on the 7-segment display.

3.3 `snake_game_tb.v` – integration testbench

- **Scenario 1 - crash without eating** The snake drives straight into a wall without eating any food. The FSM transitions correctly (IDLE → PLAY → GAME_OVER) and the length stays at 1 throughout.

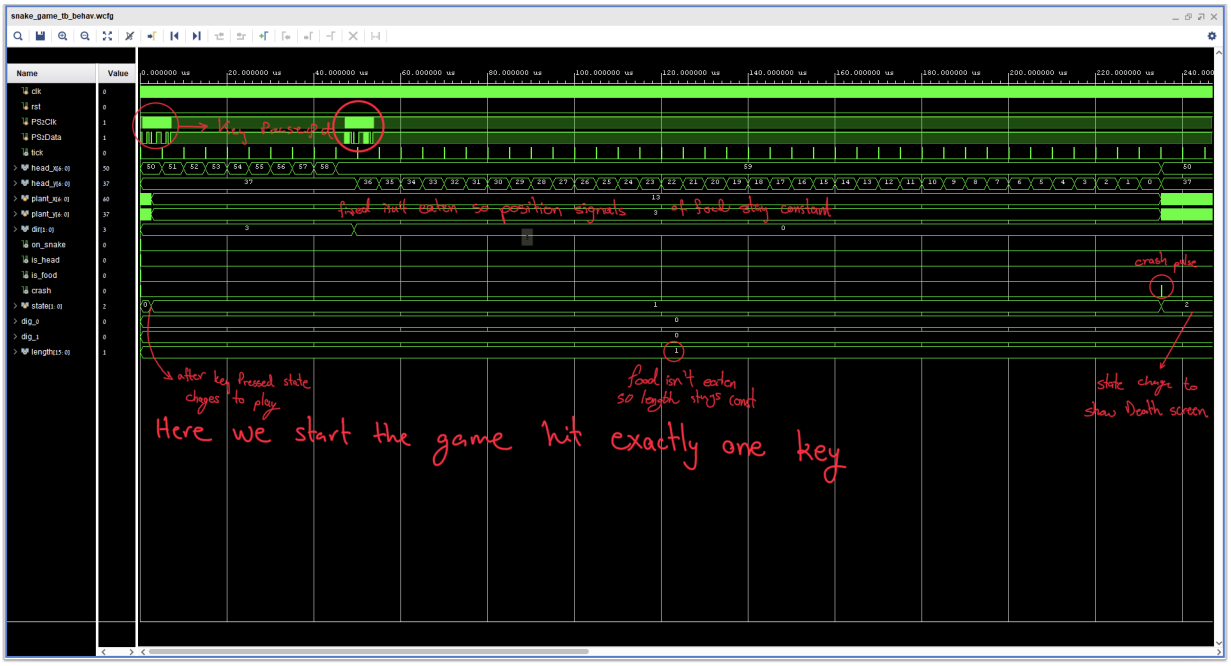


Figure 3: Scenario 1 - crash without eating

- **Scenario 2 - eat then crash** The snake eats one food item, growing to length 2 and confirming that `snake.v`'s growth logic works, then crashes into a wall.

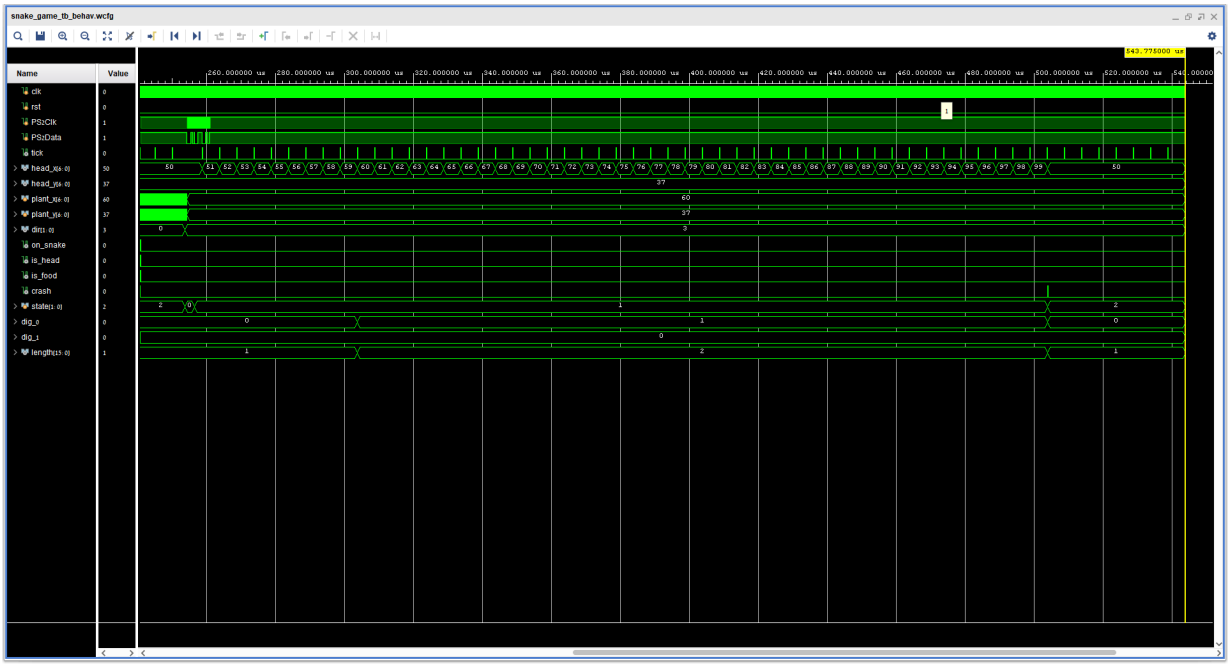


Figure 4: Scenario 2 - eat then crash

4 Internal Game Logic

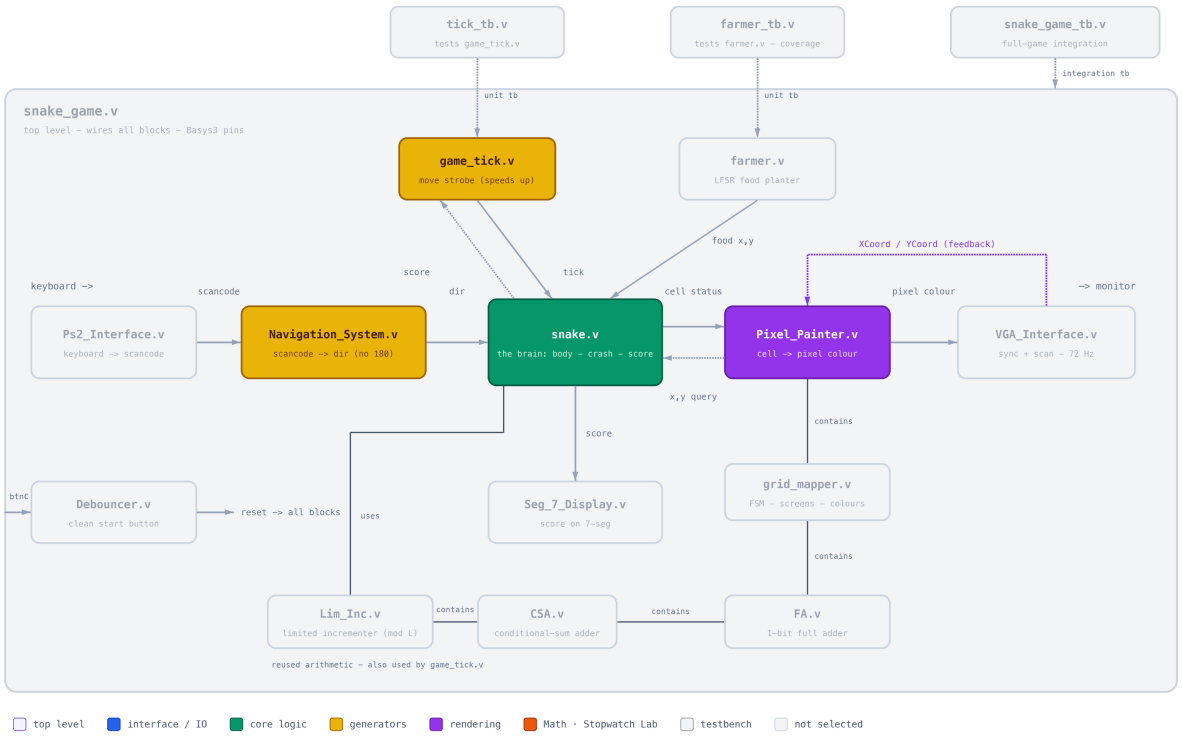


Figure 5: Internal logic modules highlighted.

4.1 `snake.v` - the game core

Purpose. Owns the snake body, movement, growth, collision detection and score.

Table 3: `Snake` interface (parameters `GRID_X`, `GRID_Y`).

Port	Dir	Width	Description
`clk`, `reset`	in	1	Clock and synchronous reset.
`tick`	in	1	Movement strobe from `Game_Tick`.
`dir`	in	2	Direction from `Navigation_System`.
`keyPressed`	in	1	Entropy for the food LFSR.
`x`, `y`	in	7/7	Cell queried by the renderer.
`on_snake`, `is_head`, `is_food`	out	1	Answers to the query: body / head / food at (x, y).
`crash`	out	1	Latched game-over flag.
`score`	out	16	Current score (= snake length).

Internal design. The body is stored as two coordinate register arrays `body_x/body_y[0..63]` (`MAX_LEN`= 64) with the head at index 0. Movement is a shift: on every tick each element copies its predecessor and the head takes the freshly computed next position. Combinational logic computes the next head from `dir`, and a 64-way comparison loop checks whether the next head lands on the body (self collision) or the query cell lies on the body (render query). To relax timing, the next-head, self-hit and eat signals are registered one clock before being consumed (`next_x_r`, `hit_self_r`, `is_eaten_r`, `tick_d`). A crash is latched when the next head leaves the grid or hits the body; growth happens when the next head equals the food cell. The food position proposed by `farmer` is latched into `plant_x/plant_y` only when the

food is eaten (or at reset), so the food stays put between meals.

Verification. Exercised in full by `snake_game_tb` (movement, steering, growth, wall crash, restart); the per-tick log prints head position and length so the shift behaviour is directly observable.

4.2 `farmer.v` - food generator

Purpose This module serves as a pseudo-random food coordinates generator for the food position on the board (`food_x`, `food_y`).

Internal design It holds a 16-bit shift register that shifts left every clock (`100 MHz clk`), and the LSB becomes the `Xor` of the `lfsr[15], lfsr[13], lfsr[12], lfsr[10], keyPressed` the Farmer generates output that looks random but it raility is fully deterministic so we choose to make the shift every clock and use the `keyPressed` as an entropy source to make the output less predictable. The food coordinates are computed as follows: `food_x` = `lfsr`[6:0] mod `GRID_X` and `food_y` = `lfsr`[13:7] mod `GRID_Y`.

Verification The testbench clocks the shift register 15,000 times (≈ 2 proposals per grid cell), toggles `keyPressed` randomly (≈ 1% of clocks) and records every proposed coordinate.

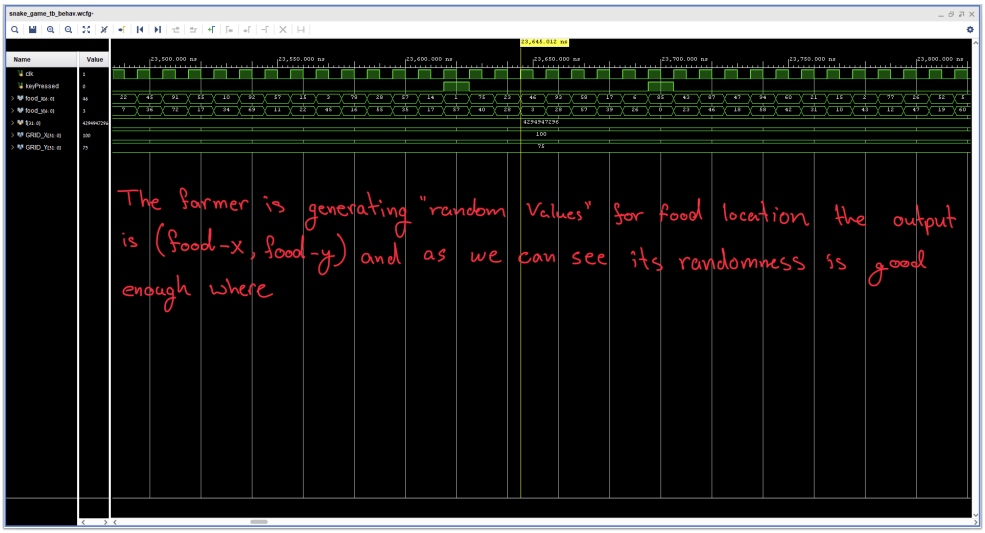


Figure 6: Food-placement coverage recorded by `farmer_tb` (15,000 samples on the 100 × 75 grid; green = proposed at least once, dark = never proposed).

The map also makes the *modulo bias* of the generator visible: as `food_x` is a 7-bit value (0–127) reduced mod 100, columns 0–27 are proposed twice as often – the denser left band in the figure; likewise rows 0–52 for `food_y` (mod 75). Every cell of the grid is reachable, and the bias is invisible during play, so this was accepted as a good trade-off for such a cheap generator.

Coverage Figure 7 shows the resulting coverage of the 100×75 grid: green cells were proposed at least once, dark cells never(we have achived ≈ 81% coverage) so we can confidently say that the farmer is good enogh to generate food coordinates.

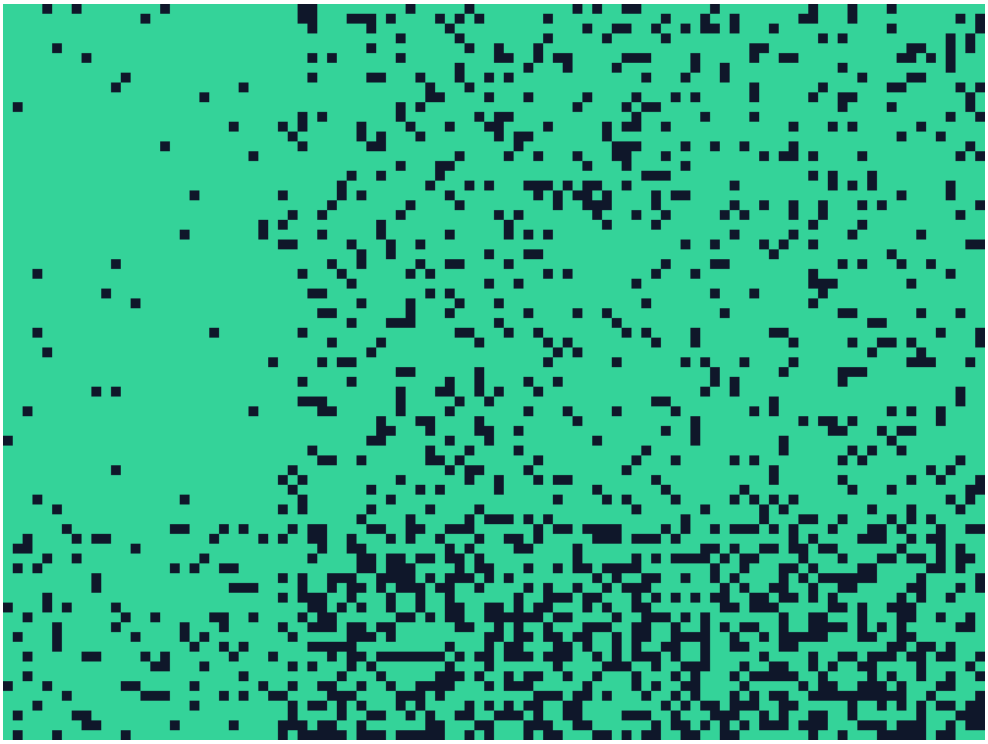


Figure 7: Food-placement coverage recorded by `farmer_tb` (15,000 samples on the 100×75 grid; green = proposed at least once, dark = never proposed).

5 Timing Block

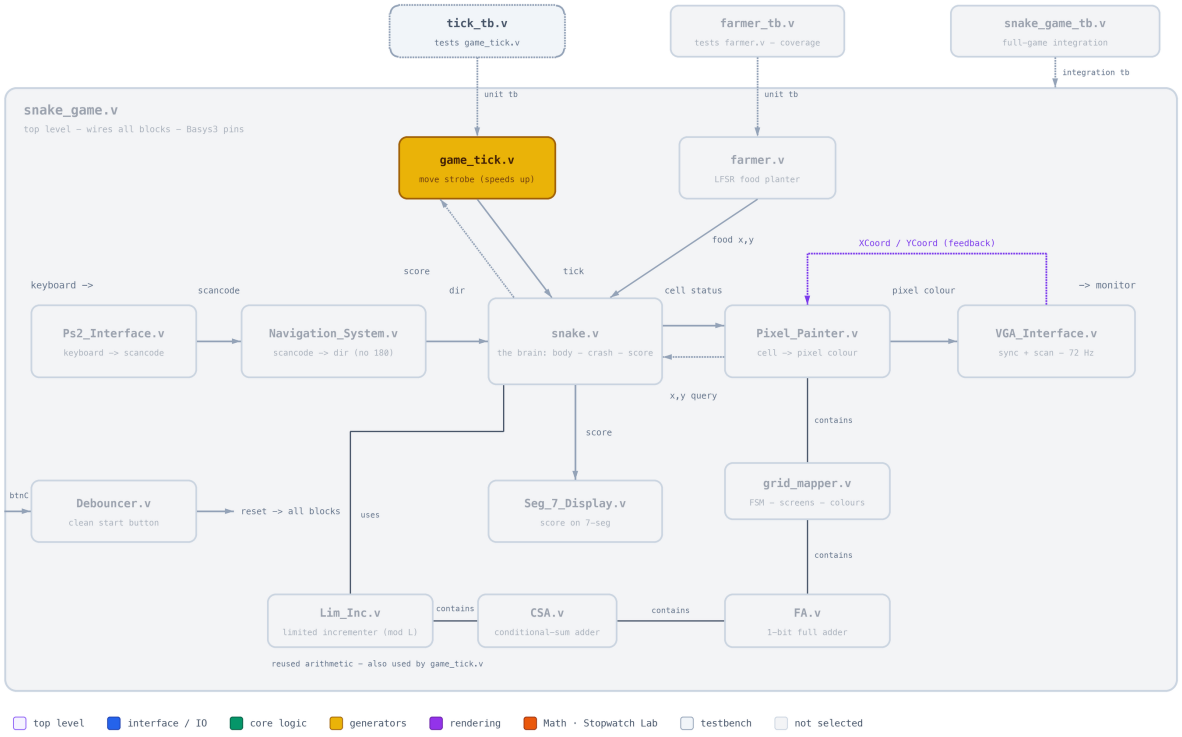


Figure 8: Timing block highlighted.

5.1 game_tick.v

Purpose. Generates the movement strobe and implements the difficulty ramp: the higher the score, the faster the snake.

Internal design. A counter compares against a programmable period `tick_speed` and emits a one-clock `tick` pulse on wrap-around. The period starts at the parameter `TICK_MAX` (14,285,714 clocks = 7 Hz at 100 MHz) and decreases linearly with the score:

$$\texttt{tick\_speed} = \texttt{TICK\_MAX} - \texttt{score} \cdot 2^{18},$$

clamped after 54 steps so the period never underflows. The module is fully parametric in `TICK_MAX` (the testbench and top level both instantiate it explicitly with their own value).

Table 4: Tick rate vs. score (at 100 MHz, `TICK_MAX`= 14,285,714).

Score	1	10	20	30	40
Tick rate	7.1 Hz	8.6 Hz	11.1 Hz	15.6 Hz	26.3 Hz

Verification – `tick_tb.v`. The unit testbench sweeps the score through $0, 10, 50, 60, 64, 80$ (crossing the clamp) and exposes the internal period plus a derived frequency probe (`hz = 100 MHz / tick_speed`) so the ramp and the clamping are directly visible in the waveform.

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Annotated waveform of `tick_tb`: arrows on the shrinking distance between `tick` pulses as `score` steps up, and on `tick_speed` saturating once the clamp (score ≥ 54) is reached.

6 Rendering Block

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Rendering block highlighted.

6.1 Pixel_Painter.v

Purpose. Bridges between the pixel domain (VGA scan coordinates) and the cell domain (game grid), and composes the final pixel color.

Internal design. The scan position is divided by 8 (`XCoord >> 3`) to obtain the queried grid cell (x, y), and by 4 to obtain the 200×150 bitmap coordinates used by the splash screens. The cell color returned by `GridMapper` is overlaid with black 1-pixel grid lines (every 8th row/column) while the game is in PLAY; `start_game` (i.e. "the game is on the PLAY screen") is exported to the top level, where it releases the snake core from reset.

6.2 `grid_mapper.v`

Purpose. The screen FSM and color oracle: decides *what is shown* for every cell in each game phase.

Internal design. A three-state FSM – IDLE (welcome screen), PLAY, GAME_OVER – advances on `keyPressed` and `crash`. Two 200×150 monochrome bitmaps are preloaded with `$readmemb` into block RAM: the welcome splash and the skull "YOU DIED" screen (Figure 9). In PLAY, a priority `casez` paints head, food, body and finally the background checkerboard, whose odd/even parity is computed by the reused full adder (`FA`) as $x_0 \oplus y_0$.

Table 5: Color palette (12-bit RGB) in `grid_mapper.v`.

Element	Color	Element	Color
Snake head	`0x222`	Background even	`0x0C0`
Snake body	`0x444`	Background odd	`0x0F0`
Food	`0xF11`	Grid lines	`0x000`
Skull	`0xEEC`	"YOU DIED" text	`0xE11`
Welcome bright	`0x6E2`	Welcome dark	`0x021`

Figure 9: The two 200×150 bitmap screens rendered by `GridMapper`: welcome splash (left, IDLE state) and game-over screen (right, GAME_OVER state) – the implemented bonus requirement.

6.3 `FA.v`

Purpose. The classic 1-bit full adder from the first experiment. Here its sum output ($a \oplus b \oplus c_{in}$, with $c_{in} = 0$) computes the checkerboard parity of a cell from $x[0]$ and $y[0]$ – a small, deliberate reuse of the earliest building block of the course.

Verification. Verified exhaustively in Lab 1; its effect is the visible checkerboard in every gameplay screenshot.

7 Output Block

[PLACEHOLDER – RESULT NOT YET AVAILABLE]

Output block highlighted.

7.1 `VGA_Interface.v`

Purpose. Generates VESA 800 × 600 @ 72 Hz timing, drives the RGB pins with the color supplied by the renderer, and exports the scan coordinates.

Table 6: VGA timing (800 × 600 @ 72 Hz, 50 MHz pixel clock).

	Visible	Front porch	Sync	Back porch	Total
Horizontal (pixels)	800	56	120	64	1040
Vertical (lines)	600	37	6	23	666

Internal design. Instead of deriving a divided 50 MHz clock, the module runs entirely at 100 MHz with a `pix_en` clock-enable that fires every second cycle – a single clock domain, which keeps timing analysis clean (see Section 11). Counters `h_count`/`v_count` generate `Hsync`/`Vsync` and are exported as `XCoord`/`YCoord`; RGB is blanked outside the visible region.

Verification. Reused from Lab 6 where it was verified against the monitor and in simulation; here it is exercised by `snake_game_tb` and by the live display.

7.2 `Seg_7_Display.v`

Purpose. Shows the 16-bit score on the 4-digit 7-segment display.

Internal design. A free-running divider (`clkdiv[19:18]`) multiplexes the four digits at ≈95 Hz per digit refresh; each 4-bit nibble is decoded to segments by a truth table, and one active-low anode is selected at a time. The score is displayed in hexadecimal nibbles, which is sufficient for the achievable snake lengths.

Verification. Reused unchanged from the first experiments; verified live – the display increments each time the snake eats.

8 Utilization of Previous Labs

A central goal of this experiment was to leverage previous work. Table 7 summarises what was taken and how it was adapted.

Table 7: Reused modules and concepts.

Module	From	Adaptation for the Snake game
`FA.v`	Lab 1	Unchanged; wired as a 1-bit XOR to compute the background checkerboard parity.
`Seg_7_Display.v`	Lab 1	Unchanged; input vector connected to the live score instead of a stopwatch count.
`Debouncer.v`	Lab 1	Unchanged; conditions the reset button.
`Ps2_Interface.v`	Lab 5	Unchanged reuse of the keyboard receiver, including break-code handling – the game only consumes `scancode`/`keyPressed`.
`VGA_Interface.v`	Lab 6	Adapted: the color is now an input (`pixel_color`) computed by the new renderer, and the scan coordinates are exported so the renderer can answer "what color is this pixel?" on the fly.

Beyond whole modules, concepts were reused as well: `Game_Tick` is the counter/frequency-divider pattern from the stopwatch experiments, `GridMapper`'s screen selector applies the FSM methodology from the earlier labs, and the testbench techniques (`$display` logging, VCD dumps, plusargs) follow the flow used throughout the course.

9 Vivado Synthesis and Implementation Results

The design was synthesised and implemented in Vivado for the Basys 3 board (Artix-7 `xc7a35tlcpg236-2L`). Synthesis and implementation complete with no errors, no DRC violations, and all timing constraints met. The Vivado reports are reproduced below.

9.1 Design summary

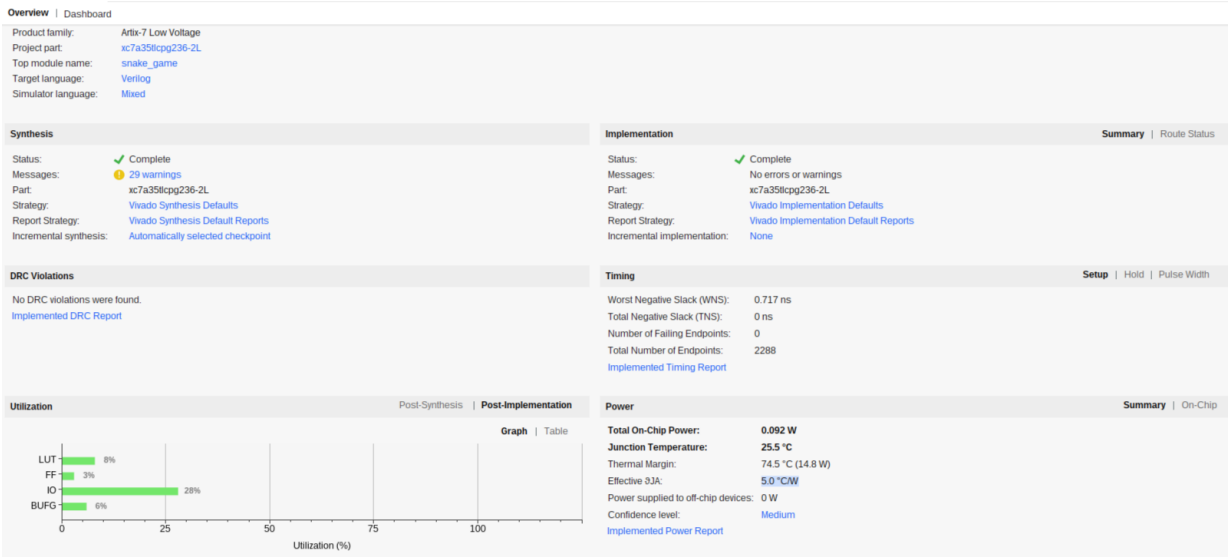


Figure 10: Project dashboard: device, synthesis/implementation status, timing, utilization and power at a glance.

9.2 Resource utilization

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	1739	20800	8.36
FF	1132	41600	2.72
IO	30	106	28.30
BUFG	2	32	6.25

Figure 11: Post-implementation utilization on the `xc7a35t`.

9.3 Timing

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.717 ns	Worst Hold Slack (WHS):	0.178 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	2288	Total Number of Endpoints:	2288	Total Number of Endpoints:	1134

All user specified timing constraints are met.

Figure 12: Design timing summary — WNS = 0.717 ns, all constraints met.

Name	Slack	Levels	High Fanout	From	To	Total ...	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 21	0.717	8	89	vga_interface/h_count_reg[6]/C	vga_interface/vgaGreen_reg[3]/D	9.334	1.842	7.492	10.0	sys_clk_pin	sys_clk_pin
Path 23	0.973	8	89	vga_interface/h_count_reg[6]/C	vga_interface/vgaRed_reg[2]/D	9.077	1.842	7.235	10.0	sys_clk_pin	sys_clk_pin
Path 22	0.947	8	64	snake/length_reg[4]/C	vga_interface/vgaBlue_reg[2]/D	9.065	2.388	6.677	10.0	sys_clk_pin	sys_clk_pin
Path 26	1.247	8	64	snake/length_reg[4]/C	vga_interface/vgaGreen_reg[1]/D	8.840	2.146	6.694	10.0	sys_clk_pin	sys_clk_pin
Path 25	1.227	8	89	vga_interface/h_count_reg[6]/C	vga_interface/vgaGreen_reg[0]/D	8.820	1.842	6.978	10.0	sys_clk_pin	sys_clk_pin
Path 24	1.139	8	65	navigation_system/dir_reg[0]/C	snake/hit_self_r_reg/D	8.816	2.390	6.426	10.0	sys_clk_pin	sys_clk_pin
Path 27	1.853	9	148	vga_interface/v_count_reg[7]/C	vga_interface/vgaGreen_reg[2]/D	8.178	2.278	5.900	10.0	sys_clk_pin	sys_clk_pin
Path 28	1.893	8	78	vga_interface/h_count_reg[8]/C	vga_interface/vgaBlue_reg[0]/D	8.159	2.358	5.801	10.0	sys_clk_pin	sys_clk_pin
Path 30	2.049	9	148	vga_interface/v_count_reg[7]/C	vga_interface/vgaBlue_reg[1]/D	8.043	2.278	5.765	10.0	sys_clk_pin	sys_clk_pin
Path 29	1.949	8	78	vga_interface/h_count_reg[8]/C	vga_interface/vgaBlue_reg[3]/D	8.009	2.384	5.625	10.0	sys_clk_pin	sys_clk_pin

Figure 13: Worst (critical) paths report.

9.4 Power

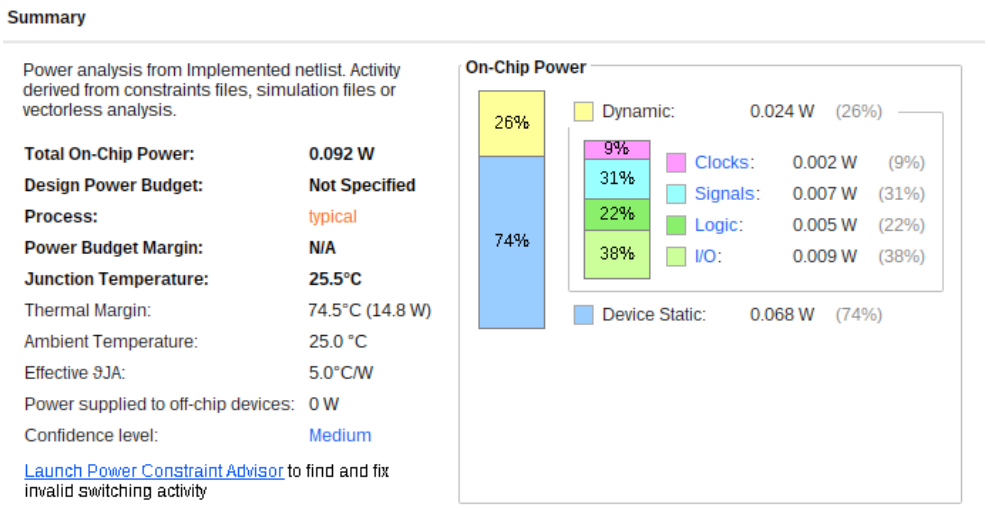


Figure 14: On-chip power estimate — total 0.092 W.

9.5 Device view

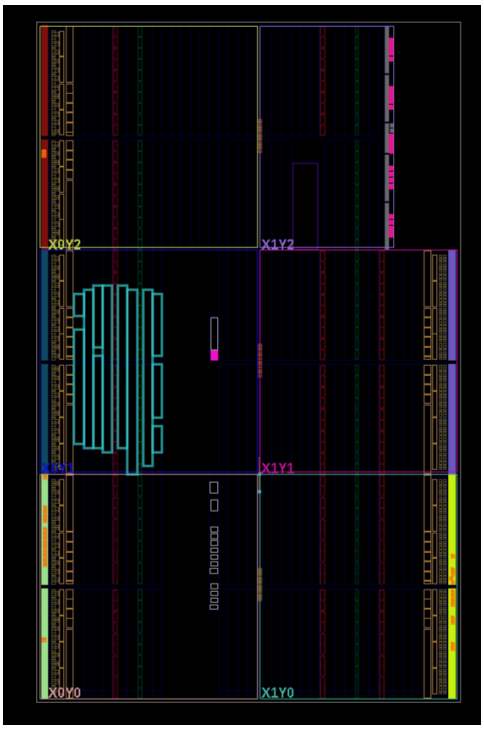


Figure 15: Implemented design placed and routed on the device.

9.6 Schematics

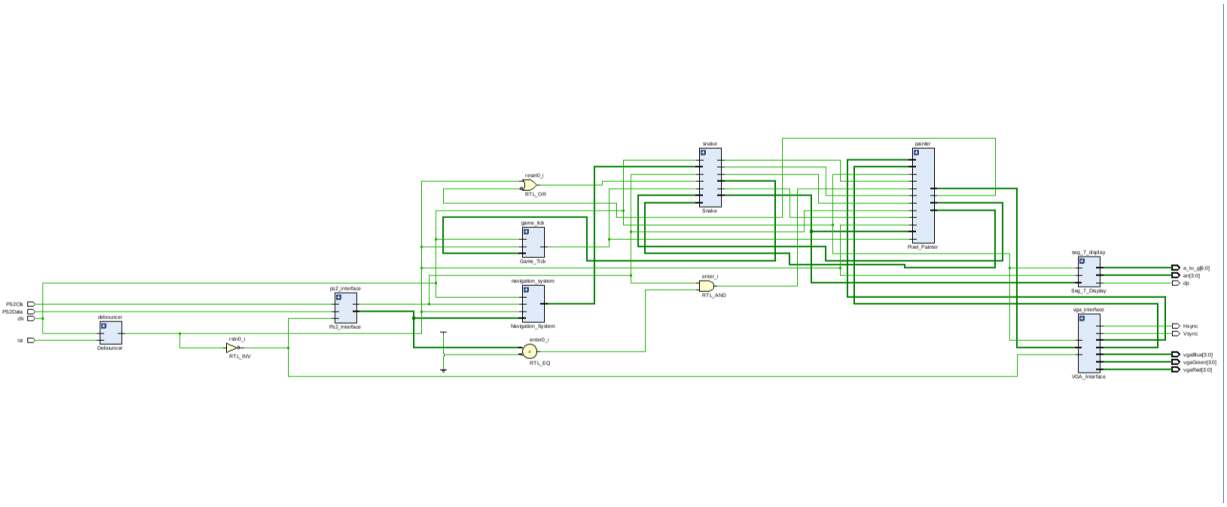


Figure 16: Elaborated RTL schematic (module-level view).

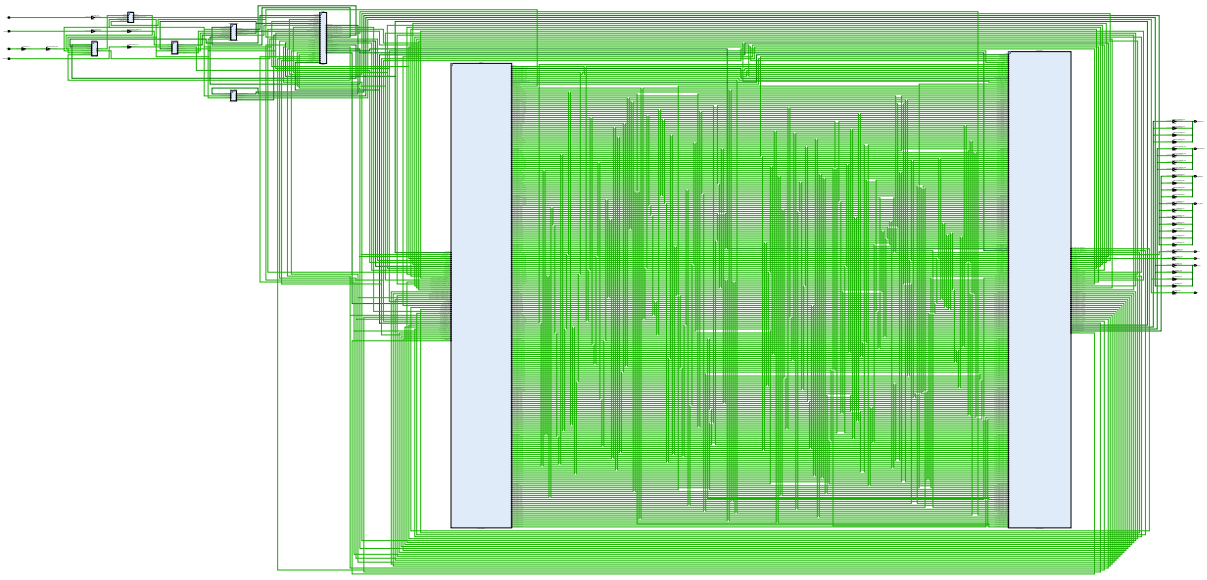


Figure 17: Synthesised netlist schematic (full design).

10 Results

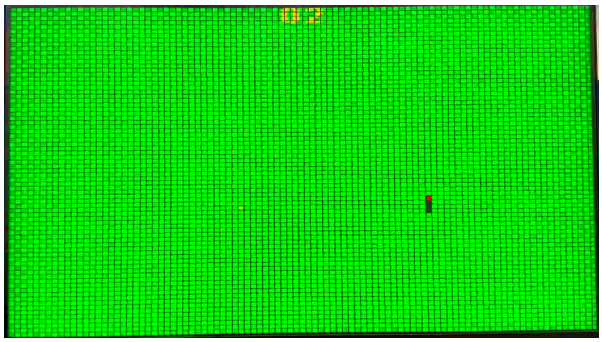
10.1 Final state of the game

The game is fully functional in simulation and on the board: it boots to the welcome splash, any key starts a round, the snake moves at 7 Hz and accelerates with every food item, the score counts up on the 7-segment display, reversal attempts are ignored, and hitting a wall or the body freezes play and shows the skull game-over screen (bonus requirement), from which a key press returns to the welcome screen. The simulated scenario in `snake_game_tb` reproduces this full lifecycle.

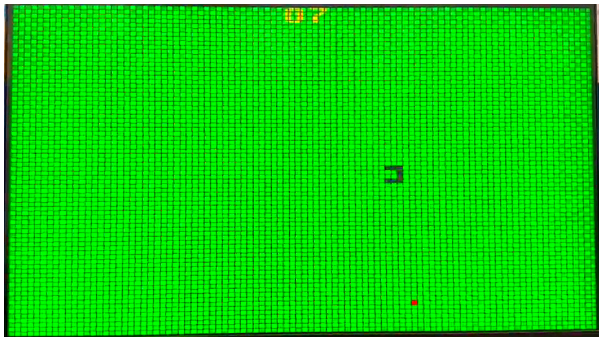
10.2 Photos of the game in action

A full round on the monitor, from start to game over, is shown in Figure 18.

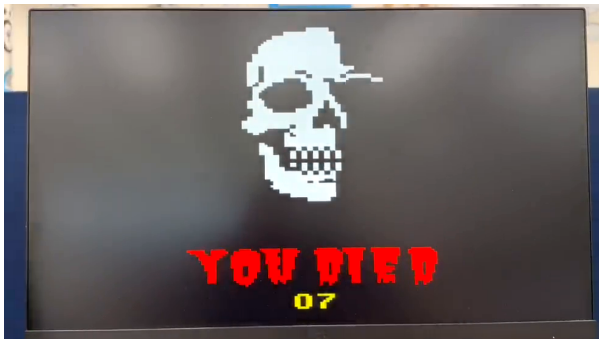
(a) Welcome / start screen



(b) Playing — snake and red food



(c) Later in the round — longer snake



(d) Game over — "YOU DIED"

Figure 18: The Snake game running on the monitor: start, play, and the game-over screen after the snake collides.

10.3 Demonstration video

A video of the game running on the Basys3 board is available on YouTube:

https://youtu.be/REPLACE-ME

[REPLACE THE LINK ABOVE — set `\youtubelink` at the top of this file to your YouTube URL]

11 Problems Encountered and Solutions

1. **Debouncer X-propagation in simulation.** The debouncer's hysteresis counter has no reset, so in simulation it powers up as `X` and the internal reset pulse never resolves (on the FPGA the global reset initialises it). *Solution:* the integration testbench forces the internal `reset` net directly for a few cycles instead of pressing the button.

2. **The real game tick is far too slow to simulate.** At 7 Hz, one snake step takes 14.3 million clocks. *Solution:* `snake_game_tb` generates its own fast tick (every 500 clocks) and forces it onto the internal `tick` net, so a whole game fits in a short simulation.

3. **Keyboard break codes re-triggered keys.** A PS/2 key release sends `F0` + scancode, which naively looks like a second key press. *Solution:* the receiver skips `E0`/`F0` bytes and re-arms on the byte following `F0`, so exactly one `keyPressed` fires per physical press.

4. **Timing warnings from a divided pixel clock.** Driving logic from a register-divided 50 MHz clock produced "non-clocked sequential cell" critical warnings in Vivado. *Solution:* the VGA module was restructured to run at 100 MHz with a `pix_en` clock-enable every second cycle – one clock domain, no derived clocks.

5. **Long combinational paths in the body scan.** Comparing the next head against all 64 body slots combinationally, directly at the tick, created long paths. *Solution:* the next-head coordinates, self-hit and eat flags are registered one cycle before the movement update consumes them (`next_x_r`, `hit_self_r`, `is_eaten_r`).

6. **Food generator bias.** The coverage run (Figure 7) exposed the modulo bias of the LFSR mapping. Rather than adding a rejection loop, the bias was measured, shown to leave every cell reachable, and accepted; keyboard entropy was mixed into the LFSR feedback to decorrelate games.

7. **[PLACEHOLDER – RESULT NOT YET AVAILABLE]**
   Any additional problems encountered during the in-lab session and their solutions.

12 Conclusion

The experiment achieved all mandatory requirements and the bonus: a playable, accelerating Snake game with VGA graphics, keyboard control, hardware score display, and dedicated welcome and game-over screens. The design cleanly separates reused interface blocks from the new game logic, is parametric in the grid size and game speed, and was verified bottom-up – unit testbenches for the new generator modules, a coverage analysis for the food placement, and a full-game integration testbench that walks the complete IDLE → PLAY → GAME_OVER lifecycle.